



NAND-GATE BASED HDL IMPLEMENTATION

Implementation of NOT16, AND16, OR16, MUX16, OR8WAY, MUX4WAY16, MUX8WAY16,
DMUX4WAY and DMUX8WAY Using Only NAND Gates

Project Report

This report follows the same NAND-only, NAND2Tetris-style HDL approach as the supplied reference report, extending it to wider and multi-way digital components.

Components Included

16-bit gates: NOT16, AND16, OR16, MUX16

Multi-input / routing gates: OR8WAY, MUX4WAY16, MUX8WAY16, DMUX4WAY, DMUX8WAY

For every component: purpose, NAND-based construction, HDL code, code description, and truth table / functional table.

1. NOT16 Gate Using NAND Gates

NOT16 has a 16-bit input and produces the bitwise complement. Each bit is inverted independently with one NAND gate used as a NOT gate.

Functional expression: $Y[i] = \text{NOT}(A[i])$ for $i = 0 \dots 15$.

HDL Code

```
CHIP Not16 {
  IN in[16];
  OUT out[16];
  PARTS:
    Not(in=in[0], out=out[0]);
    Not(in=in[1], out=out[1]);
    Not(in=in[2], out=out[2]);
    Not(in=in[3], out=out[3]);
    Not(in=in[4], out=out[4]);
    Not(in=in[5], out=out[5]);
    Not(in=in[6], out=out[6]);
    Not(in=in[7], out=out[7]);
    Not(in=in[8], out=out[8]);
    Not(in=in[9], out=out[9]);
    Not(in=in[10], out=out[10]);
    Not(in=in[11], out=out[11]);
    Not(in=in[12], out=out[12]);
    Not(in=in[13], out=out[13]);
    Not(in=in[14], out=out[14]);
    Not(in=in[15], out=out[15]);
}
```

Code description

Each input bit is sent to a NOT operation. In the NAND-only hierarchy, the Not chip is itself implemented as Nand(a=in, b=in, out=out). Therefore all 16 output bits are the complements of the corresponding input bits.

Truth Table / Functional Table

Input bit A	Output Y
0	1
1	0

2. AND16 Gate Using NAND Gates

AND16 performs bitwise AND on two 16-bit buses.

Functional expression: $Y[i] = A[i] \text{ AND } B[i]$ for $i = 0 \dots 15$.

HDL Code

```
CHIP And16 {
  IN a[16], b[16];
  OUT out[16];
  PARTS:
    And(a=a[0], b=b[0], out=out[0]);
    And(a=a[1], b=b[1], out=out[1]);
    And(a=a[2], b=b[2], out=out[2]);
    And(a=a[3], b=b[3], out=out[3]);
    And(a=a[4], b=b[4], out=out[4]);
    And(a=a[5], b=b[5], out=out[5]);
    And(a=a[6], b=b[6], out=out[6]);
    And(a=a[7], b=b[7], out=out[7]);
    And(a=a[8], b=b[8], out=out[8]);
    And(a=a[9], b=b[9], out=out[9]);
    And(a=a[10], b=b[10], out=out[10]);
    And(a=a[11], b=b[11], out=out[11]);
    And(a=a[12], b=b[12], out=out[12]);
    And(a=a[13], b=b[13], out=out[13]);
    And(a=a[14], b=b[14], out=out[14]);
    And(a=a[15], b=b[15], out=out[15]);
}
```

Code description

Sixteen independent 1-bit AND operations are performed. Each And chip can be built from two NAND gates: first NAND the inputs, then NAND the result with itself.

Truth Table / Functional Table

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

3. OR16 Gate Using NAND Gates

OR16 performs bitwise OR on two 16-bit buses.

Functional expression: $Y[i] = A[i] \text{ OR } B[i]$ for $i = 0 \dots 15$.

HDL Code

```
CHIP Or16 {
  IN a[16], b[16];
  OUT out[16];
  PARTS:
    Or(a=a[0], b=b[0], out=out[0]);
    Or(a=a[1], b=b[1], out=out[1]);
    Or(a=a[2], b=b[2], out=out[2]);
    Or(a=a[3], b=b[3], out=out[3]);
    Or(a=a[4], b=b[4], out=out[4]);
    Or(a=a[5], b=b[5], out=out[5]);
    Or(a=a[6], b=b[6], out=out[6]);
    Or(a=a[7], b=b[7], out=out[7]);
    Or(a=a[8], b=b[8], out=out[8]);
    Or(a=a[9], b=b[9], out=out[9]);
    Or(a=a[10], b=b[10], out=out[10]);
    Or(a=a[11], b=b[11], out=out[11]);
    Or(a=a[12], b=b[12], out=out[12]);
    Or(a=a[13], b=b[13], out=out[13]);
    Or(a=a[14], b=b[14], out=out[14]);
    Or(a=a[15], b=b[15], out=out[15]);
}
```

Code description

Each bit pair uses an OR chip. The OR chip is NAND-only: invert each input with NAND-as-NOT and NAND the two inverted signals.

Truth Table / Functional Table

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

4. MUX16 Using NAND Gates

MUX16 selects one of two 16-bit inputs using one select bit. When sel=0, a is selected; when sel=1, b is selected.

Functional expression: $out[i] = (a[i] \text{ AND NOT } sel) \text{ OR } (b[i] \text{ AND } sel)$.

HDL Code

```
CHIP Mux16 {
  IN a[16], b[16], sel;
  OUT out[16];
  PARTS:
    Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
    Mux(a=a[1], b=b[1], sel=sel, out=out[1]);
    Mux(a=a[2], b=b[2], sel=sel, out=out[2]);
    Mux(a=a[3], b=b[3], sel=sel, out=out[3]);
    Mux(a=a[4], b=b[4], sel=sel, out=out[4]);
    Mux(a=a[5], b=b[5], sel=sel, out=out[5]);
    Mux(a=a[6], b=b[6], sel=sel, out=out[6]);
    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
    Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);
}
```

Code description

Sixteen 1-bit MUX operations share the same select signal. The underlying Mux is implemented from NAND gates by creating NOT sel, generating two active-low paths, and combining them with a final NAND.

Truth Table / Functional Table

sel	Selected output
0	a
1	b

5. OR8WAY Using NAND Gates

OR8WAY combines eight single-bit inputs and outputs 1 if at least one input is 1.

Functional expression: $out = a \text{ OR } b \text{ OR } c \text{ OR } d \text{ OR } e \text{ OR } f \text{ OR } g \text{ OR } h$.

HDL Code

```
CHIP Or8Way {
  IN in[8];
  OUT out;
  PARTS:
    Or(a=in[0], b=in[1], out=t01);
    Or(a=in[2], b=in[3], out=t23);
    Or(a=in[4], b=in[5], out=t45);
    Or(a=in[6], b=in[7], out=t67);
    Or(a=t01, b=t23, out=t0123);
    Or(a=t45, b=t67, out=t4567);
    Or(a=t0123, b=t4567, out=out);
}
```

Code description

The eight inputs are ORed in pairs, the four pair-results are ORed into two results, and the final two results are ORed. Every Or operation is NAND-only according to the reference construction.

Truth Table / Functional Table

Number of 1s in input	out
0	0
1 or more	1

6. MUX4WAY16 Using NAND Gates

MUX4WAY16 selects one of four 16-bit inputs using two select bits. sel=00 selects a, 01 selects b, 10 selects c, and 11 selects d.

Functional expression: $out = Mux16(Mux16(a,b,sel[0]), Mux16(c,d,sel[0]), sel[1])$.

HDL Code

```
CHIP Mux4Way16 {
  IN a[16], b[16], c[16], d[16], sel[2];
  OUT out[16];
  PARTS:
    Mux16(a=a, b=b, sel=sel[0], out=ab);
    Mux16(a=c, b=d, sel=sel[0], out=cd);
    Mux16(a=ab, b=cd, sel=sel[1], out=out);
}
```

Code description

Two MUX16 chips first choose within the pairs (a,b) and (c,d). A third MUX16 selects between those pair-results. Since Mux16 is NAND-based, the complete component is constructed from NAND gates through the hierarchy.

Truth Table / Functional Table

sel[1]	sel[0]	Selected input
0	0	a
0	1	b
1	0	c
1	1	d

7. MUX8WAY16 Using NAND Gates

MUX8WAY16 selects one of eight 16-bit inputs using three select bits.

Functional expression: sel=000..111 selects a..h respectively.

HDL Code

```
CHIP Mux8Way16 {
  IN a[16], b[16], c[16], d[16],
    e[16], f[16], g[16], h[16], sel[3];
  OUT out[16];
  PARTS:
    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=abcd);
    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=efgh);
    Mux16(a=abcd, b=efgh, sel=sel[2], out=out);
}
```

Code description

Two MUX4WAY16 blocks select within the first four and last four inputs. A final MUX16 uses the most significant select bit to choose between those groups.

Truth Table / Functional Table

sel[2:0]	Selected input
000	a
001	b
010	c
011	d
100	e
101	f
110	g
111	h

8. DMUX4WAY Using NAND Gates

DMUX4WAY routes one input to exactly one of four outputs using two select bits.

Functional expression: sel=00 -> a, 01 -> b, 10 -> c, 11 -> d.

HDL Code

```
CHIP DMux4Way {
  IN in, sel[2];
  OUT a, b, c, d;
  PARTS:
    DMux(in=in, sel=sel[1], a=low, b=high);
    DMux(in=low, sel=sel[0], a=a, b=b);
    DMux(in=high, sel=sel[0], a=c, b=d);
}
```

Code description

The first DMux divides the input into two groups according to sel[1]. Each group is then divided according to sel[0]. The underlying DMux uses NAND gates for select inversion, path generation, and output inversion.

Truth Table / Functional Table

sel[1]	sel[0]	a	b	c	d
0	0	in	0	0	0
0	1	0	in	0	0
1	0	0	0	in	0
1	1	0	0	0	in

9. DMUX8WAY Using NAND Gates

DMUX8WAY routes one input to exactly one of eight outputs using three select bits.

Functional expression: sel=000..111 routes the input to a..h respectively.

HDL Code

```
CHIP DMux8Way {
  IN in, sel[3];
  OUT a, b, c, d, e, f, g, h;
  PARTS:
    DMux(in=in, sel=sel[2], a=low, b=high);
    DMux4Way(in=low, sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=high, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

Code description

A first 2-way DMux divides the signal into two groups. Each group is then routed through a DMux4Way. Because DMux and DMux4Way are NAND-based, the complete 8-way demultiplexer follows the NAND-only design hierarchy.

Truth Table / Functional Table

sel[2:0]	Active output
000	a
001	b
010	c
011	d
100	e
101	f
110	g
111	h

10. Complete Gate Summary and Verification

The table below summarizes the requested components and their NAND-based construction strategy.

Component	Inputs / Select	Main NAND-based idea
NOT16	16-bit in	16 independent NAND-as-NOT operations
AND16	16-bit a,b	16 independent NAND-based AND operations
OR16	16-bit a,b	16 independent NAND-based OR operations
MUX16	16-bit a,b + sel	16 NAND-based 2-way MUX operations
OR8WAY	8 single-bit inputs	Tree of NAND-based OR operations
MUX4WAY16	4 × 16-bit + 2 sel	Three NAND-based MUX16 stages
MUX8WAY16	8 × 16-bit + 3 sel	Two MUX4WAY16 + final MUX16
DMUX4WAY	1-bit in + 2 sel	Three NAND-based DMux stages
DMUX8WAY	1-bit in + 3 sel	One DMux + two DMux4Way stages

Verification procedure

1. Create each HDL file using the code provided in this report.
2. Ensure the lower-level chips (Not, And, Or, Mux and DMux) are available as NAND-based implementations, following the supplied reference report.
3. Run the chips in the Nand2Tetris Hardware Simulator.
4. Apply all relevant truth-table or functional-table combinations.
5. Confirm that the simulator outputs match the expected behavior shown in this report.

Important HDL note

The supplied reference report uses `NAND(a=..., b=..., out=...)` as the primitive and builds higher-level components structurally. This extended report preserves that hierarchical approach: wider and multi-way chips instantiate lower-level NAND-based chips. Thus, the complete design can ultimately be reduced to NAND primitives.

Conclusion

These nine components extend the NAND-only design method from single-bit gates to 16-bit operations, multi-input OR logic, and multi-way data routing. The HDL code demonstrates structural hardware composition, while the truth and functional tables provide a direct method for verification.